OPENCLASSROOMS
PROJET 7
AI ENGINEER

Un assistant intelligent pour recommander des événements culturels

POC RAG pour Puls-Events : LangChain, Mistral, FAISS, FastAPI

Kseniia Sazonova

Le besoin : répondre juste, sans inventer

Le problème

Un LLM seul ne connaît pas la programmation de Paris en octobre 2026 et invente des événements plausibles.

Une recherche par mots-clés ne comprend pas « une sortie gratuite pour les enfants le mercredi ».

La réponse : le RAG

Retrieval : retrouver les événements pertinents dans une base à jour.

Generation : rédiger la réponse à partir de ces événements uniquement, avec titre, dates, lieu, tarif et lien.

Objectifs du POC

Faisabilité : la chaîne complète tourne sur une machine standard, services gratuits.

Valeur métier : réponses utilisables et honnêtes.

Performance mesurée : jeu de test annoté, métriques, régressions détectées.

Périmètre : Paris, agenda de la Ville

2 762
événements récupérés
25 sept. au 31 déc. 2026

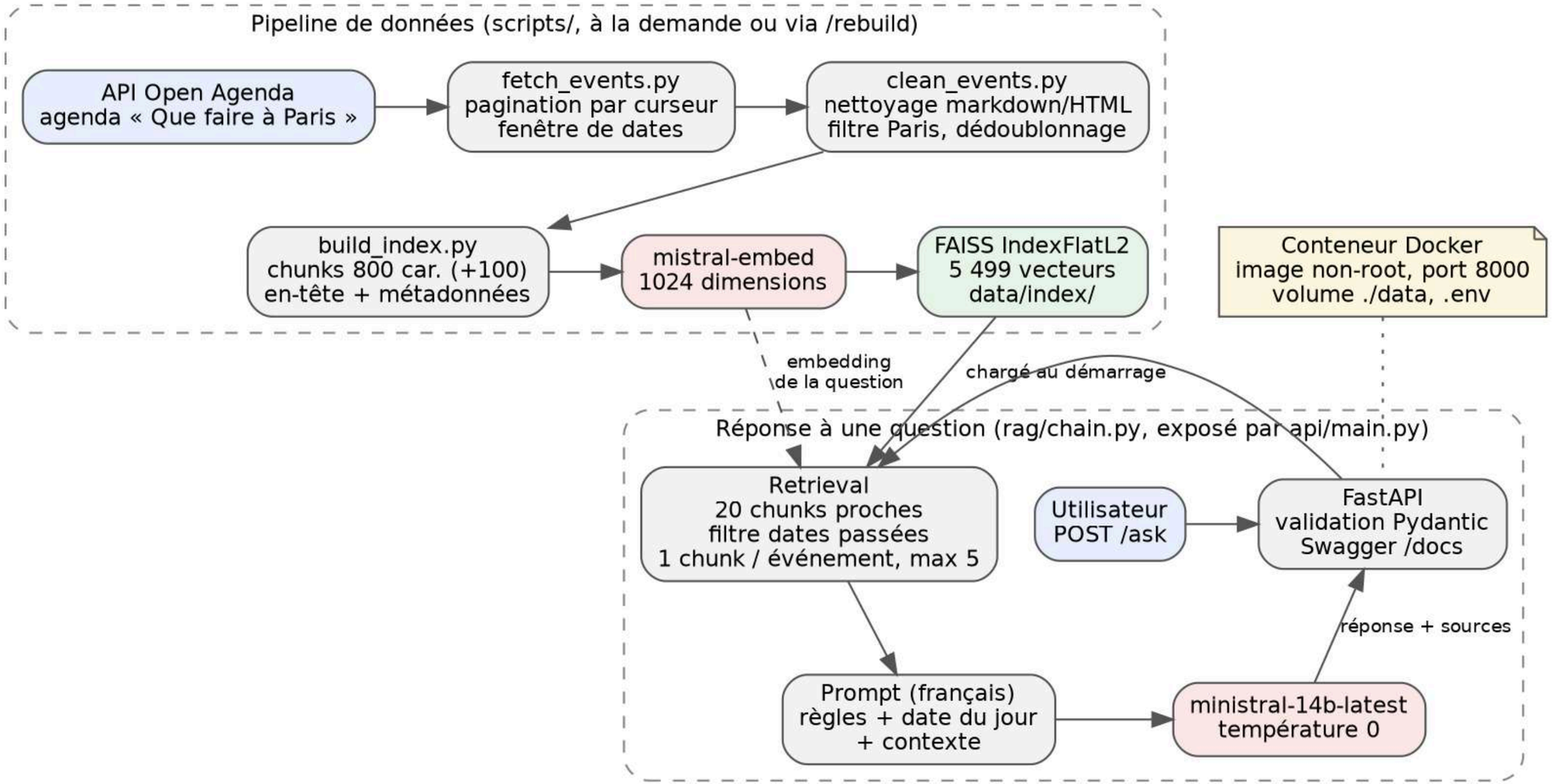
2 650
conservés après nettoyage
(Paris intra-muros)

5 499
chunks de 800 caractères
vectorisés

1 024
dimensions par vecteur
(mistral-embed)

- Source : API Open Agenda, agenda « Que faire à Paris » (uid 648405), fiches complètes et cohérentes.
- Fenêtre : au moins une séance entre aujourd'hui et trois mois ; paramètre de la route /rebuild.
- Hors périmètre : banlieue (100 fiches), fiches sans description (12), vie nocturne commerciale absente de la source.

Architecture



- Pipeline : collecte, nettoyage, chunks, embeddings, index FAISS persisté.
- Question : API FastAPI, retrieval, prompt, ministral-14b, réponse + sources.
- Index et modèle chargés une fois au démarrage ; /rebuild recharge à chaud.

Des fiches brutes à une base vectorielle

1

Collecte

Pagination par curseur, fenêtre de dates, detailed=1 pour la description longue.

2

Nettoyage

Markdown et HTML retirés (131 fiches sur 200), français, Paris, dédoublonnage, dates aplaties.

3

Chunking

800 caractères, recouvrement 100, coupé sur paragraphes puis phrases. En-tête titre, dates, lieu, tarif.

4

Embeddings

mistral-embed, 1 024 dimensions, lots sous 16 000 tokens, quelques minutes pour tout l'index.

5

Index FAISS

IndexFlatL2, recherche exacte, moins d'une milliseconde à cette taille. Métadonnées à côté de chaque vecteur.

6

Persistance

index.faiss et index.pkl dans data/, hors Git, remplacés à chaque reconstruction.

Modèles : Mistral, offre gratuite

mistral-embed

Embeddings de 1 024 dimensions, même modèle pour les chunks et pour la question. Environ 1,2 M de tokens pour l'index complet : quelques centimes au tarif public.

ministral-14b-latest

Génération, température 0. Le plus capable des modèles ouverts sur l'offre gratuite : mistral-small et medium y sont à 0 requête par minute.

- Critères : imposé par la mission (Mistral), français natif, intégration LangChain officielle, coût nul.
- Prompt en français : données, questions et réponses sont en français ; mélanger les langues coûte à un petit modèle.
- Limite assumée : un 14B raisonne mal sur le calendrier. Réponse : calculer en Python ce qu'il calcule mal.

La chaîne RAG en quatre étapes

1

Retrieval

20 chunks les plus proches de la question (FAISS).

2

Filtrage en Python

Événements terminés écartés, un chunk par événement, 5 événements maximum.

3

Prompt

Règles, date du jour, week-end et saison calculés, mois couverts par chaque événement.

4

Génération

ministral-14b rédige ; réponse + sources (uid, titre, dates, lieu, URL).

Ce que dit le prompt

Répondre uniquement à partir du contexte.

Donner titre, dates, lieu, tarif.

S'il n'y a pas de correspondance exacte : le dire, puis proposer les événements les plus proches en précisant la différence.

Ne jamais inventer d'événement, de date ou de lieu.

Les phrases de calendrier ne sont ajoutées que si la question contient une expression relative (« ce week-end », « en novembre »).

API REST, conteneur, intégration continue

Route	Rôle
GET /health	Vérification de vie (health check Docker)
POST /ask	Question en entrée, réponse + sources en sortie, validation Pydantic
POST /rebuild	Reconstruit l'index sur une fenêtre de dates et le recharge ; clé X-API-Key

- FastAPI pour la validation et Swagger (/docs), utilisable sans code par les équipes produit.
- 48 tests unitaires hors ligne, un test fonctionnel contre l'API réelle.

Docker

Image Python 3.12 + dépendances verrouillées (uv.lock), utilisateur non-root, health check. Secrets et données hors de l'image : .env et volume ./data.

GitHub Actions

Tests à chaque push et pull request, branche main protégée. Évaluation complète à la demande sur un instantané d'index versionné, résultats en artefact.

Démonstration

docker compose up, puis trois questions via l'API :

1. « Quels concerts de jazz ce week-end ? »
2. « Une activité gratuite pour les enfants ? »
3. « Y a-t-il un opéra de Wagner ce mois-ci ? » (rien ne correspond)

Évaluation : jeu annoté, quatre métriques

Jeu de test annoté

14 questions couvrant les thèmes de l'agenda et plusieurs formes de contrainte temporelle.

12 questions « recommandation » : réponse de référence rédigée à la main + identifiants des événements attendus.

2 questions « sans correspondance » (opéra de Wagner, cuisine japonaise) : le système doit refuser d'inventer.

Évaluation datée au 25 septembre 2026, sur un instantané d'index versionné : reproductible.

Métriques

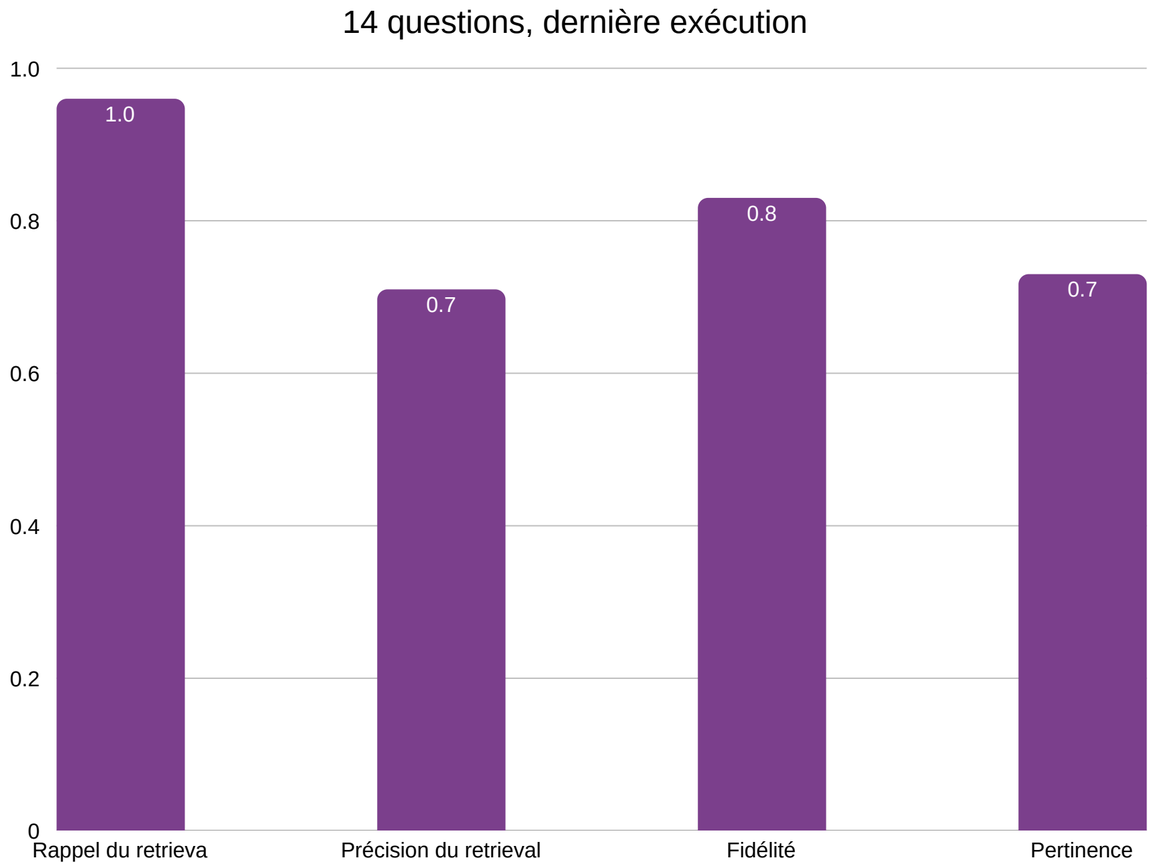
Rappel et précision du retrieval : exactes, calculées sur les identifiants annotés, sans juge.

Fidélité (Ragas) : chaque affirmation de la réponse est-elle soutenue par le contexte ? Détecte les hallucinations.

Pertinence (Ragas) : la réponse traite-t-elle la question ?

Juge : ministral-14b. Les métriques Ragas de contexte ont été abandonnées : ce juge les note mal ; les identifiants annotés répondent à la même question exactement.

Résultats sur 14 questions



Exécution	Fidélité	Pertinence
1. base (10 chunks, t = 0,2)	0,83	0,77
2. 20 chunks, filtrage des dates passées	0,68	0,75
3. saison dans chaque prompt, t = 0	0,55	0,73
4. calendrier seulement si question datée, quasi-correspondances	0,83	0,73

- Rappel 0,96 : presque tous les événements attendus sont retrouvés.
- L'exécution 3 : un ajout de prompt anodin a coûté 0,28 de fidélité ; l'évaluation l'a vu, l'exécution 4 l'a corrigé.

Ce que l'évaluation a révélé, et corrigé

Défaut observé	Correction
Un événement du 29 sept. au 31 janv. n'a pas lieu « en octobre » selon le modèle	Mois couverts listés en Python pour chaque événement
Jours du week-end mal calculés	Dates du week-end calculées et énoncées dans le prompt
Question sans date restreinte au week-end à venir	Calendrier injecté seulement si la question contient une expression relative
Mot-clé « Jeunes » non associé aux adolescents	Vocabulaire des mots-clés expliqué
Refus de proposer des quasi-correspondances	Signaler l'absence exacte, puis proposer le plus proche
Un dimanche soir, concert du samedi encore proposé	Événements terminés filtrés en Python avant le prompt
Réponses variables d'une exécution à l'autre	Température 0

Limites du POC et prochaines étapes

Limites

Couverture : un seul agenda, riche en culture publique, pauvre en vie nocturne.

Temporalité : la période demandée n'est pas filtrée avant la recherche.

Modèle : un 14B commet encore des erreurs ; 30 requêtes par minute.

Évaluation : juge de 14B, consignes en anglais, une exécution isolée est bruitée.

Fraîcheur : index instantané, à reconstruire.

Prochaines étapes

Retrieval sensible aux dates : extraire la période demandée et filtrer avant la recherche.

Plusieurs agendas Open Agenda fusionnés, puis d'autres sources.

Modèle plus grand (mistral-small ou large) pour la génération et le juge.

Mise à jour incrémentale de l'index, historique de conversation.

Production : déploiement managé, secrets gérés, authentification sur /ask, expériences Langfuse.

Streamlit UI

Un POC complet, mesuré, reproductible

Pipeline, index, chaîne RAG, API, Docker, CI, évaluation, observabilité.
Dépôt : github.com/ash0nyx/rag-event-assistant-oc-7, version v0.2.0.

Merci

Je suis prête à répondre à vos questions.